

Gunmandu Party: An AI Reader for Korean Web Novelists

UChicago CSMC 33231 Final Project

Summer Han
University of Chicago
hangoeun@uchicago.edu

Abstract

In *Oldboy*, the protagonist is locked in a room for fifteen years with only dumplings (gunmandu) and Welch's grape soda. Korean web novel readers joke about doing the same to their favorite authors. A common comment is "do you like gunmandu?" — meaning the reader loves the work so much they would lock the writer in a room, feed them gunmandu and Welch's, and not let them out until the story is done. The joke misses that most authors are already in that room, drafting twenty or fifty episodes in silence before a single reader arrives. **GunmanduParty** is a writing tool that fills that silence with simulated readers. Instead of generating prose for the author or critiquing their draft, it generates comments — the short, emotional, unstructured reactions real readers leave on web novel platforms like Joara. The personas are built from my own corpus of 804 reader comments, clustered into four archetypes. To test whether these archetypes capture community-level patterns rather than work-specific noise, I ran a cross-work transfer test: three of four archetypes transfer to a separate work at cosine similarity above 0.85. This report documents what I built, the dead ends along the way, and what cross-work validation revealed about the gap between sounding like a reader and using a reader's actual words.

1. Motivation

On Joara, a major Korean web novel platform, a free-serializing writer must publish at least 20 episodes to reach the "Today's Best" ranking — the only discoverability slot that matters. To win the engagement metrics that drive the ranking, writers stockpile episodes and drop five at a time near the threshold. On Naver Series and Munpia, writers are expected to release about 50 additional episodes when launching paid serialization. The result: every Korean web novel writer, regardless of experience level, ends up writing in silence for months before a single reader arrives. It is built into the platform structure itself.

Existing AI writing tools miss this. Sudowrite and NovelAI write prose; the writer here needs readers, not more writing. AutoCrit's AI Beta Reader gives structured critique through reader personas, but critique tells the writer what to *fix* — what they need during stockpiling is a sign someone is reading. I-eum AI, a Korean attempt at sentence continuation and feedback,

has shut down. Reza et al. (2025), in a systematic review of 109 HCI papers and interviews with 15 writers, found that creative writers in particular value control over *translating* (the act of writing) and *reviewing* (the act of judging one's own writing) — and they categorize existing AI writing support into four strategies: structured guidance, guided exploration, active co-writing, and critical feedback. None of the four involve simulating a reader. Gero et al. (2025), in interviews with 33 creative writers, identified the *human element* as a core principle: writers see their work as fundamentally written *by a person, to be read by another person*, and view LLM text generation as "mimicry" that misses the intended human audience. GunmanduParty answers both findings in an asymmetric way — the AI stays on the *reader* side, simulating the missing human audience rather than displacing the human writer's act of translating or reviewing. The line that matters is **feedback ≠ comments**. Feedback is editorial; comments are emotional. Comments are what keeps a writer writing.

2. Design Rationale

The project has one thesis: **AI as a reader, not a writer**. Reading is what the tool simulates; writing stays with the writer. Two design principles follow:

1. AI does not write. The system never writes prose for the manuscript, never autocompletes, never suggests phrasings, never touches the writer's text. The editor pane is sacred. AI output lives in a separate panel, set apart from the manuscript in look and function.

2. AI does not critique. The system writes reader-style comments, not editorial critique. Where AutoCrit gives lengthy feedback from a user-defined persona explaining why readers are likely to lose track or get confused, GunmanduParty's emotional reactor says "으앙 ㅠㅠ 이해가 잘 안되요 ㅠㅠ ㅠㅠ" (sob TT I can't understand TTT) — a real comment from the corpus. The output mirrors what real readers actually write — sometimes short and emotional, sometimes long and analytical, but always reactive rather than prescriptive. Comments cost the writer nothing; they motivate. Critique costs effort to read and apply, and can drain the very momentum the system is meant to preserve.

These two principles are constraints, not features. Everything I built had to fit inside them.

GunmanduParty does two things: it lets writers write, and it makes writing fun again.

3. Intended Users and Use Contexts

Primary user: a Korean web novel writer drafting episodes before any real reader has seen them. Four sub-profiles, each asking a different question of the virtual readers:

- **The Beginner** — does my voice work at all?
- **The Emerging Writer** — would anyone care if they found this?
- **The Newly Launching Writer** — where is the cliffhanger that hooks?
- **The Established Writer** — will my old readers come back for this one?

Use context.

- **When.** During stockpiling — the weeks or months between writing Episode 1 and a real reader reading Episode 1. Not for post-publication. Once real readers arrive, virtual readers step aside.
- **Workflow.** Per-episode. The writer finishes a chapter, pastes it in, clicks [읽히기], reads the reactions. Moves on.

- **Frequency.** Episode-by-episode is the main use. A secondary stuck-mode use — throwing in a half-finished scene to see if it lands — is supported but not tuned for.
- **Not for.** Line-editing, copyediting, structural critique, marketability checks. The two principles in §2 rule these out by construction.

4. Prototype

GunmanduParty is a web app: a React frontend, a FastAPI backend, and the Claude API for inference. Reader personas come from a personal corpus of 804 Joara comments processed offline. The backend loads them at startup along with a 1,024-dimensional BGE-M3 embedding index over the same corpus, used for retrieval when generating an author-defined persona.

Four backend endpoints:

Endpoint	Purpose
GET /api/personas	Return four-persona metadata (3 core + 1 author-defined slot)
POST /api/episodes/read	Generate comments from one persona for one episode
POST /api/episodes/read/all	Run all personas concurrently
POST /api/personas/analyze	Turn a structured author description into a full persona

Table 1. GunmanduParty backend API endpoints.

Four personas in the UI:

-  감정 추론러 (Emotional Reactor) — short, intense emotional reactions
-  분석적 충성팬 (Devoted Analyst) — long, theory-building, foreshadowing-aware
-  정중한 응원자 (Polite Supporter) — brief, courteous per-episode check-ins
-  사용자 정의 (Author-Defined) — built from a structured author description

Each persona writes 2–3 comments per episode, capped at 10 per episode total (with optional anonymous patterns) to match a real comment section.

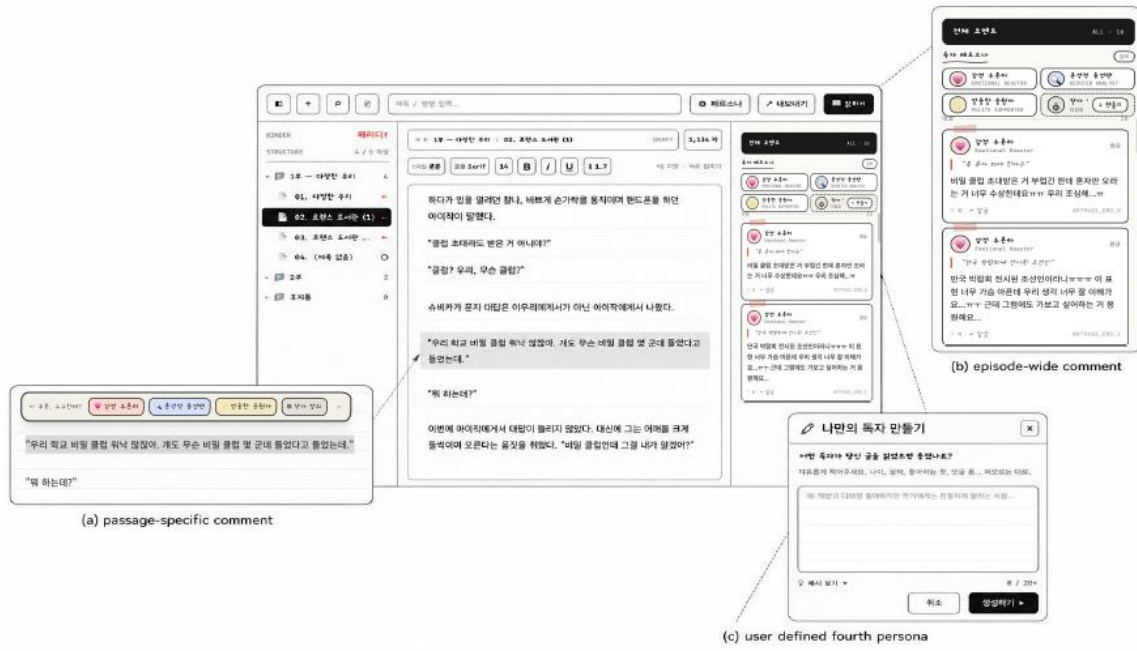


Figure 1: GunmanduParty helps writers feel read while drafting in silence: (a) drag any passage to summon a specific reader; (b) episode-wide reactions appear as a mixed comment stream; (c) writers describe their ideal reader, and the system grounds it in real comment data.

5. Key Design Decisions

Decision	Alternatives	What I Chose	Why
System category	Formal editorial feedback; persona-based critique	Comment simulation	Comments solve the motivation problem; critique solves a different one (correctness) and costs effort to apply
Persona source	Generic LLM personas; crowdsourced data	My own 804-comment corpus	Authentic Joara tone; controllable; I own the data; validation possible
Number of personas	3 hypothesized in proposal (emotional / plot-sensitive / worldbuilding)	4 found through clustering, 3 surfaced	Plot-sensitive and worldbuilding readers form one "Devoted Analyst" archetype; a "Polite Supporter" archetype I had not predicted emerged; a "Hiatus" archetype did not fit the drafting use case
Author-defined persona input	Structured form; predefined persona menu	Free-form text (with example prompts shown)	Lets writers describe their ideal reader in their own words; the system handles classification under the hood
Combine vs. Novel mode	One LLM call generates everything	Two-stage: Claude scores similarity to	Combine stays 100% data-grounded; Novel is grounded

Decision	Alternatives	What I Chose	Why
		existing archetypes → Python mixes (Combine) OR Claude writes fresh with retrieval grounding (Novel)	by corpus vocabulary and retrieval, not LLM imagination
Novel grounding source	NVIDIA Nemotron-Personas-Korea (evaluated and rejected); no grounding	Work A corpus only (vocab pool + BGE-M3 top-8 retrieval)	Nemotron is a persona/demographic profile — it describes <i>who readers are</i> , not <i>how readers comment</i> . Platform demographics could in principle substitute, but mapping them to comment behavior is its own project, and outliers are hard to handle. Work A's corpus already contains the behavioral signal I need.
Work B integration	Another Joara romance fantasy work of comparable size, reader count, and engagement pattern	Hold Work B out (my own second work); validation only	Scraping another writer's comments would touch their rights and consent; my second work is comparable in genre and platform, and I have the right to use it. Cross-work transfer is the strongest evidence the project has; touching Work B would destroy it.

Table 2. Key design decisions across the project, with alternatives considered and rationale for the final choice. The seven decisions cover system category, persona derivation, author-defined persona handling, and validation methodology.

6. Process and Iterations

The project went through several pivots. The five below mattered most.

Pivot 1: Three hypothesized personas → Four data-found archetypes (one dropped)

The proposal named three personas from my domain experience: emotional immersion, plot-sensitive, worldbuilding. After comment-level clustering (BGE-M3 + KMeans, $k=4$) and then reader-level meta-clustering on the 38 readers with five or more comments, four archetypes came out. Silhouette analysis on the reader-level clustering confirmed $k=4$ as the best fit.

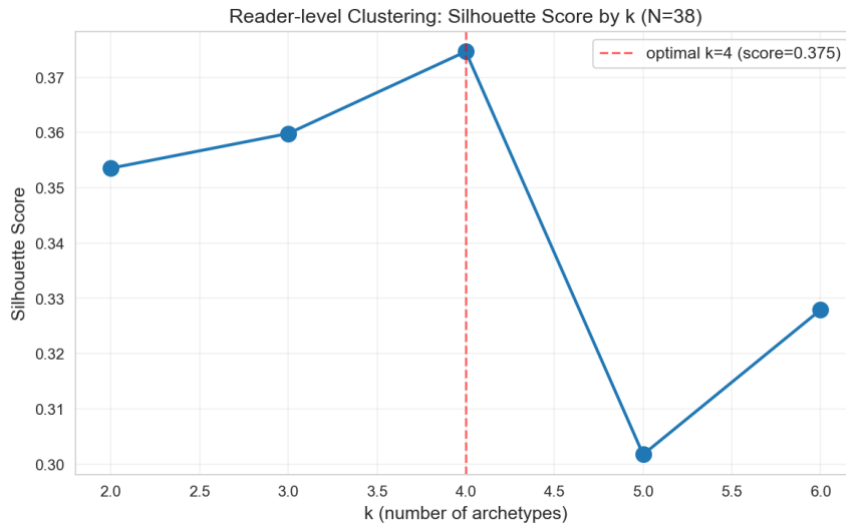


Figure 2. Reader-level clustering silhouette score by k ($N=38$ active readers). $k=4$ peaks at 0.375; $k=5$ drops sharply, $k=6$ partially recovers — indicating 4 as the natural granularity for the data.

Archetype	n	Avg length	Dominant cluster	What it is
0: Hiatus-Bound Waiters	4	24 chars	"기다림" (hiatus appeals)	Pleas for the author to resume updates during long hiatus
1: Devoted Analysts	15	77 chars	Long analysis	Theory-building + foreshadowing
2: Polite Supporters	5	44 chars	Author-directed	Short per-episode courtesy comments
3: Emotional Reactors	14	30 chars	Emotional reactions	Short intense feelings

Table 3. Reader archetypes derived from meta-clustering (work A, $k=4$, $N=38$ active readers). Each archetype is characterized by reader count (n), average comment length, dominant comment cluster, and behavioral description. Archetype 0 (Hiatus-Bound Waiters) was excluded from the production persona system on use-case grounds.

The archetype-to-cluster relationship is dominant but not exclusive: each archetype has one comment cluster it draws from most heavily, plus secondary patterns.

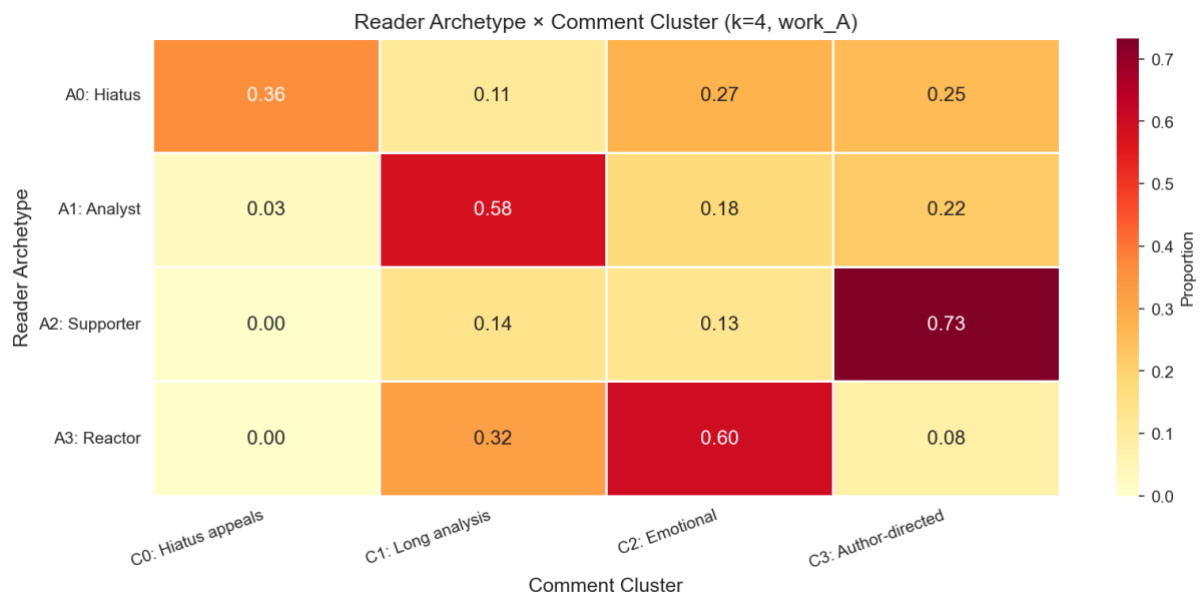


Figure 3. Reader archetype $\times$ comment cluster distribution (work A, $k=4$). Diagonal-ish pattern: each reader archetype concentrates its commenting in one cluster: A0 Hiatus $\rightarrow$ C0 Hiatus appeals (36%); A1 Analyst $\rightarrow$ C1 Long analysis (58%); A2 Supporter $\rightarrow$ C3 Author-directed (73%); A3 Reactor $\rightarrow$ C2 Emotional (60%). A2 Supporter shows the strongest dominance (73%), A0 Hiatus the most spread.

Archetype 3 confirmed my emotional-immersion hypothesis. Archetype 1 folded plot-sensitive *and* worldbuilding into one type — readers who track plot inconsistencies also track world details. They are the same people. Archetype 2 (Polite Supporter) I had not predicted; from my own experience as a reader and writer in women-targeted serialized fiction communities, it is a pattern that recurs across platforms in this genre cluster, but I had not named it as a distinct archetype. Archetype 0 (Hiatus-Bound Waiters) I dropped on use-case grounds. Its comments ("기웃기웃", "ㄱㅇㄱㅇ") are pleas for the author to resume updates during long hiatus periods. They do not fit drafting.

Median dominance across all 38 active readers was 56.9% (mean 59.4%) — readers lean toward an archetype, they do not belong to one. Only 2 of 38 readers stayed within a single cluster; the rest spread their comments across 2–4 clusters.

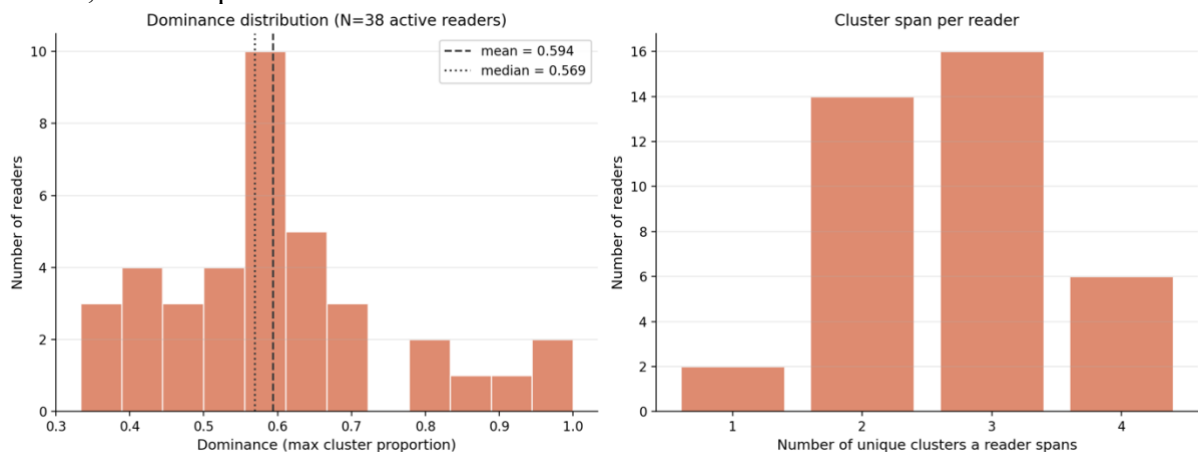


Figure 4. Left: dominance distribution across active readers ($N=38$). Mean 59.4%, median 56.9%. Right: number of unique clusters each reader's comments span. Only 2 readers stayed in a single cluster; 16 spread across 3, 6 across all 4.

What surprised me: the data both confirmed and corrected my hunches. I had seen the Polite Supporter pattern in women-targeted serialized fiction communities but had not named it as a distinct archetype. I had over-split the analyst type. The dominance finding also surprised me — I had been treating personas as exclusive identities, but the data showed even active readers drift 40% of the time. The system reflects this as 60% dominant pattern + secondary patterns.

Pivot 2: Trusting centroids → hand-picking examples

The first plan was to use each cluster's centroid — the mathematical average of its members — as the representative for that archetype. Centroids are clean to compute and standard in clustering literature.

Reading the actual comments told a different story. For some archetypes the centroid surfaced bland comments. For Devoted Analyst it did something worse: the centroid pulled in long emotional reactions ("고아아아 드디어!! 만났다!! ㅠㅠ") and polite check-ins ("잘보고갑니다") — comments that matched the *length* of an analyst comment but not its *character*. Theory-building lived further out in the cluster.

I switched to hand-picking. For each archetype I read the cluster members and selected comments that made the archetype unmistakable: long, theory-building speculation for the Analyst; intense reactions for the Reactor; short courteous check-ins for the Supporter. The selections went into `personas.json` as `example_comments`, which Claude sees in the persona prompt as few-shot anchors.

What surprised me: the standard NLP move (centroid as representative) actively hurt persona quality. A persona built on centroid examples would sound like the *average* of its archetype, which is exactly the bland, indistinguishable style I was trying to move away from. Hand-picking introduces author judgment, but it produced personas that read as distinct from each other.

Pivot 3: Author-defined persona — single LLM call → two-stage backend

The proposal described the fourth persona as "the ideal target reader the author most wants to satisfy," with the writer typing a free-form description. Benharak et al. (2024) report that even with a structured form, writers struggled to describe an imagined reader from scratch — they fell back on people they personally knew. This was the empirical reason to keep the input as free-form text rather than asking for trait-by-trait specification; the writer describes their ideal reader in whatever shape comes naturally, and the backend does the structuring.

The first version sent that description to Claude in one call to produce traits, vocabulary, and example comments all at once. Two problems: the output was opaque (hard to say what the system was doing), and it did not line up with the methodology paper's two-stage framing.

The input stays free-form. What changed is how the backend handles it:

Author description (free-form text, 10-2000 chars)

↓ Stage 1: Claude scores similarity to 3 core archetypes

↓ `_decide_mode(scores)`

├── Combine: $\text{top} \geq 0.6$

├── Stage 2a: Python mixes archetype vocab/examples weighted by scores

├── traits: "분석적 충성팬 60% + 정중한 응원자 40%"

└── Novel: $\text{top} - \text{bottom} < 0.10$ (no archetype anchors)

Stage 2b: BGE-M3 retrieves top-8 nearest real comments from

the 804-comment corpus.
Claude writes traits/vocab/examples grounded by the
base vocab pool + retrieved comments.

A stress test made this concrete. For the weak description "매일 저녁에 소설을 읽는 사람" (someone who reads novels every evening), Stage 1 produces near-flat scores (~0.34/0.33/0.33). Mode resolves to Novel. BGE-M3 pulls real comments close to "evening reading" — mostly Polite Supporter cluster ("매일 잘 보고 있어요", "오늘도 정주행"). The result speaks in the Supporter idiom, anchored to real vocabulary rather than to whatever Claude would invent. The output is weak because the input is weak — but weak in a fair, data-anchored way.

Pivot 4: Length constraint — one prompt line, +0.09 cosine

After cross-work testing against Work B's real comments, generated comments matched real ones at only 0.77 overall centroid cosine. I added one line to the persona prompt:

"길이: 한국 웹소설 댓글은 짧음. 대부분 5-40 자. 80 자 절대 넘지 마세요."

(Length: Korean web novel comments are short. Mostly 5–40 characters. Never over 80.)

The effect:

Metric	v1 (no length cap)	v2 (length cap)	Δ
Overall centroid cosine	0.771	0.862	+0.091
Per-episode mean cosine	0.640 $\pm$ 0.083	0.718 $\pm$ 0.062	+0.078
Vocab JS divergence	0.609	0.622	+0.013 (worse)
Length KS statistic	0.893	0.760	−0.133

Table 4. Effect of adding a single length-constraint sentence to the persona prompt (v1 $\rightarrow$ v2). Embedding-level similarity (cosine) improves substantially; vocabulary-level similarity (JS divergence) does not. Comparison uses work B's 56 real comments as ground truth.

What surprised me: one sentence moved embedding alignment a lot — but vocabulary divergence barely moved, and slightly got worse. The system catches *register* cheaply and *idiolect* poorly. This is the strongest case for the author-defined persona feature: a given work's reader vocabulary cannot be pulled out of a different work's corpus, no matter how much prompt engineering you do. One plausible explanation is that when forced to be short, the model fell back on its own high-prior short expressions rather than work A's reader-specific ones.

Pivot 5: Novel-mode grounding — Nemotron $\rightarrow$ corpus-only

The first plan considered NVIDIA Nemotron-Personas-Korea as a cold-start source. On inspection, Nemotron's personas are general Korean demographic profiles, not web-novel-reader profiles — wrong domain. It was rejected at evaluation and never integrated. The final pick was Work A's own corpus, used two ways: a base vocabulary pool (affect tokens, words, bigrams, phrases with length stats), and BGE-M3 retrieval of the top-K nearest real comments.

The trade-off is real. The system ties to one corpus; a different writer would need to supply their own. The mitigation is that the pipeline is reproducible — the same offline scripts produce a fresh persona set from any writer's exported comments.

Cross-work validation result

Work A archetypes were tested against Work B's 56 comments. Work B was held out from every step of persona construction. Both works are romance fantasy serialized on Joara. The two works differ sharply:

	Work A	Work B
Episodes	66	13
Writing period	2017–2019	Sept–Oct 2018 (15 days)
Comments collected	809 (804 after preprocessing)	56
Active readers (5+ comments)	38	0

Table 5. Work A vs work B: corpus characteristics. Work A served as the basis for persona construction; work B was held out for cross-work validation. The two works differ sharply in scale and active period, providing a meaningful transfer test rather than near-duplicate comparison.

Work B is too small for reader-level meta-clustering, but comment-level cluster centroids can still be compared.

Three of four cross-work archetype matches passed 0.85 cosine. The dropped Hiatus archetype scored the lowest row-mean similarity (0.685). Its absence in Work B's 15-day active drafting period — there was no time for a hiatus to form — gives empirical weight to the decision to drop it. This is the strongest piece of evidence the project produced.

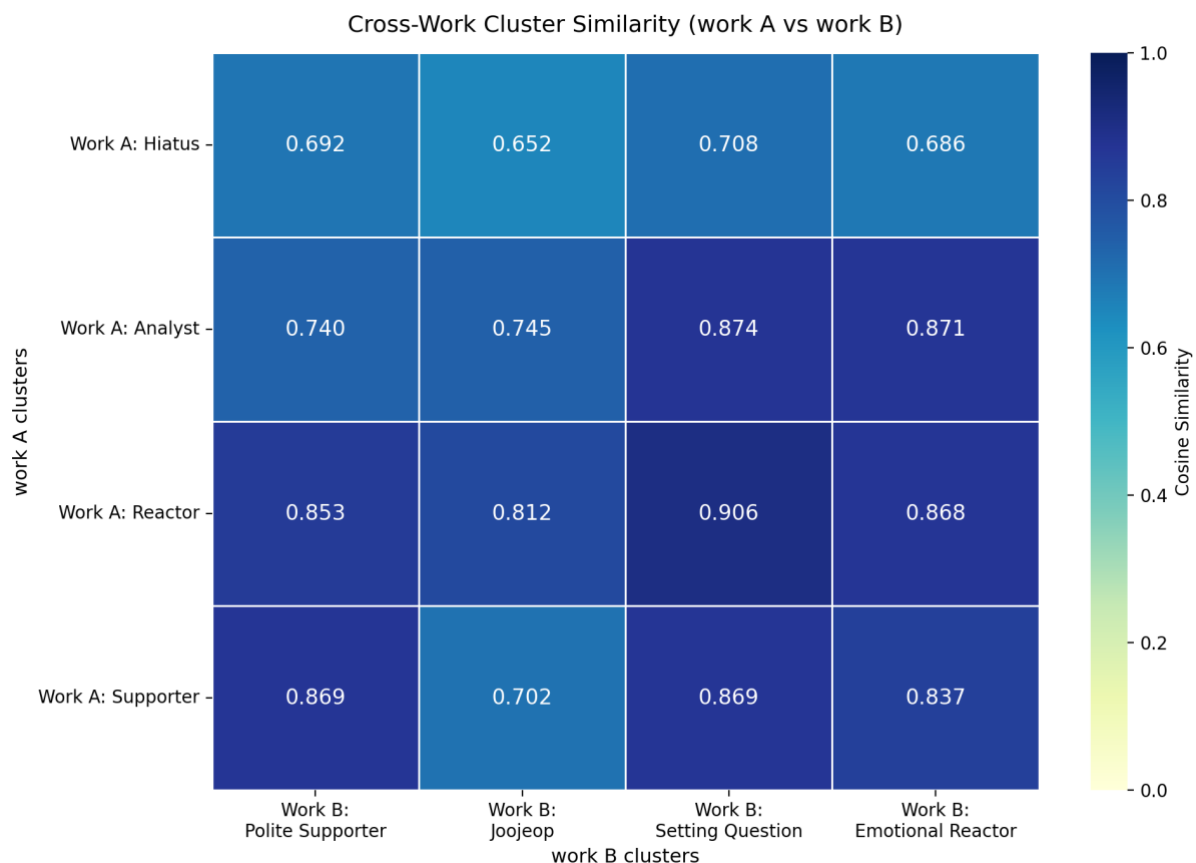


Figure 5. Cross-work Cluster Similarity matrix. The Reactor cluster is the universal attractor — every work B cluster matches it at $\cos > 0.81$, with Reactor $\leftrightarrow$ Setting Question reaching 0.906. Supporter $\leftrightarrow$ Polite Supporter confirms transfer at 0.869. The Hiatus cluster row mean = 0.685, the lowest — empirically validating its exclusion.

7. Ethical Implications

Privacy and consent. Reader comments are written by real people, even on a public platform. The corpus hashes each nickname with SHA256 and a salt at parse time; original nicknames are never written to disk. Raw comment text is never shared — only aggregate stats, methodology, and pipeline code are public. Both source works are also set to private on Joara — only I can read the novels themselves or their comment sections. This protects commenter anonymity beyond hashing, but also limits external reproducibility: no one else can independently verify the corpus. The corpus enters the system two ways: clustering produces *aggregate* persona definitions (vocabulary frequencies, length distributions, centroids), and Novel-mode retrieval shows real comments to *Claude only*, never to the user. This follows the NLP norm of releasing derived statistics, not raw text.

Data ownership. The corpus is comments on *my* work. I have a defensible (if not perfect) basis for using it. I chose this over scraping other writers' comment sections, which would raise harder consent questions.

Validation integrity. Work B's 56 comments stayed out of the system entirely — not for persona construction, not for retrieval, not for prompt tuning. The pull to fold it in was real; the right call was to keep the validation honest.

Replacement risk. The framing — *AI as reader, not writer; placeholders, not replacements* — is itself a stance. Generative AI in creative writing risks replacing the humans whose work

feeds it. By limiting the AI to reading and naming the personas as placeholders for absent real readers, the system fills a time gap (the months before real readers arrive) rather than displacing human reader-writer ties.

Limits of generalization. One writer, one platform, one cluster of genres, one language. Claims that go past this scope are not defensible from current evidence alone.

Accessibility paradox. The system's core asset — the corpus — is also its access barrier. Because the personas must come from an author's own reader comments (for the privacy, consent, and data ownership reasons above), the tool requires what it is meant to provide: existing readers. A writer with hundreds of comments can run the pipeline on their own data and use the system. A writer with zero readers — the Beginner and Emerging Writer profiles in §3, the very people the tool exists for — cannot. This is a structural cost of taking commenter consent seriously, not a solvable bug. In practice the tool currently works for one user: me.

Writer-as-developer-as-subject. I built a tool I will use on my own writing, tested with my own data, with my own corpus as ground truth. This gives strong design intuition but blurs evaluator independence. Cross-work validation helps; a multi-writer study is needed before stronger claims.

8. Reflection and Limitations

What I learned about AI. Standard NLP metrics often miss what you actually need. Three instances of the same lesson showed up across this project. First, switching from jhgan/ko-sroberta-multitask (768d, silhouette 0.055) to BAAI/bge-m3 (1024d, silhouette 0.038) *lowered* the standard clustering metric but produced much richer sub-pattern discovery (2 → 14 HDBSCAN clusters). Second, for the Analyst cluster the mathematical centroid pulled in long emotional reactions and polite check-ins instead of theory-building — length matched but character did not, and hand-picking from cluster edges captured the archetype better. Third, one line of prompt instruction about comment length moved embedding cosine alignment by +0.09 but barely touched vocabulary overlap. LLMs catch *register* easily and *idiolect* poorly; closing that gap takes grounding, not prompt engineering.

The throughline: every "standard move" I would have made by default — pick the model with higher silhouette, trust centroids as cluster representatives, treat embedding distance as a proxy for quality — was wrong for this task in a different way. Domain-fit judgment had to override metric optimization at each step.

What I learned about the task. Personas are tendencies, not identities. Median dominance across active readers was 58.9% — even active readers drift 43% of the time. Systems that treat personas as exclusive types over-fit. The right model is dominant-plus-secondary.

What I learned about myself. The triple role (writer + developer + subject) gives strong intuition, but I am not an independent judge of my own tool's quality. The cross-work test was partly me forcing myself to look at evidence I had not picked.

Limits of what I built.

- Single-author corpus. Generalization claims must stay narrow.
- Weak author descriptions resolve to Polite-Supporter-leaning Novel-mode output. The system cannot fix gaps in writer input — only ground them.
- No persona memory across chapters. A persona that loved the male lead in Chapter 3 does not remember in Chapter 4.

- No blind human authenticity test. Planned (10 friends, AI-vs-real), not run.
- Comment-level cross-work transfer is validated; reader-level archetype transfer is not (Work B too small).

With another month I would:

- Run the blind human test
- Pilot multi-author corpus integration (two or three Joara writers who consent)
- Collect comments from a different platform with consent — Munpia is the obvious next step, since its user base skews male and its dominant genres (modern fantasy, martial arts) differ from Joara's female-leaning romance fantasy core. Cross-platform corpora would test whether the archetypes generalize across reader demographics and genre conventions, or whether they are tied to Joara-specific norms.
- Add persona memory across chapters
- Start Feature 2 (Narrative Thread Tracker — proposed but cut from this submission)

Honest uncertainty. Whether the four archetypes hold across different cultural contexts. From my domain experience the patterns recur across Korean web novel platforms in the same genre cluster, but I have no evidence yet on whether equivalent archetypes appear in English-language reader communities (AO3, Royal Road, Wattpad). That is a question for future cross-cultural extension.

9. AI Disclosure

This report uses two frameworks: Fang et al.'s (FAccT 2026) procedural factors for writing, and He et al.'s (CHI 2025) AIA Statement codes for the GunmanduParty system.

Writing (Fang et al. style). AI was used with **high replaceability, high effortfulness, high intentionality, and low directness.**

- **Report.** I wrote the prose. Claude (Opus 4.7) helped outline the data sections during development and revised tone, grammar, and style on my hand-written drafts.
- **Poster.** Text compressed and polished with Claude (tone, grammar, style). Figure 1 redrawn by ChatGPT from my screenshot and prompt. Layout designed without AI.

System (He et al. style). Code authorship is split into two parts:

- **Entirely by me (no AI):** raw data scraping, parsing and anonymization, persona example selection for personas.json.
- **Collaborative [Nc, PAI]** — I described the steps and provided the data; Claude (Sonnet 4.5) wrote the implementation code; I reviewed outputs and iterated until correct: (a) Offline pipeline — clustering, archetype derivation, vocabulary extraction; (b) Runtime backend — comment generation, persona analysis, API endpoints; (c) Frontend integration.

At inference time, the runtime backend generates new content (reader comments and persona definitions) via Claude; the Stage 1 persona-classification step performs content edits rather than generation; Combine mode runs deterministically with no AI call.

Codes: **Nc** new content · **Ce** content edits · **PAI** primarily AI-generated · **HAI** human-AI blend · all flows **Pm** (prompted) and **R** (reviewed).

10. References

- [1] Benharrah, Karim, Tim Zindulka, Florian Lehmann, Hendrik Heuer, and Daniel Buschek. "Writer-Defined AI Personas for On-Demand Feedback Generation." In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, 1–18. 2024.
- [2] Gero, Katy Ilonka, Meera Desai, Carly Schnitzler, Nayun Eom, Jack Cushman, and Elena L. Glassman. "Creative Writers' Attitudes on Writing as Training Data for Large Language Models." In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pp. 1-16. 2025.
- [3] Reza, Mohi, Jeb Thomas-Mitchell, Peter Dushniku, Nathan Laundry, Joseph Jay Williams, and Anastasia Kuzminykh. "Co-writing with ai, on human terms: Aligning research with user demands across the writing process." *Proceedings of the ACM on Human-Computer Interaction* 9, no. 7 (2025): 1-37.